Réplica guiada del setup completo con acompañamiento (Bloque 3) / Tu entorno virtual, aislado y con las dependencias justas

✓ **Hecho:** Ver

📘 **UNIDAD U1.2** · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Tu entorno virtual, aislado y con las dependencias justas

Con tu repositorio ya inicializado y tu primer **commit** en **GitHub**, esta lección vuelve sobre uno de los cuatro conceptos que viste al principio de la unidad: el **entorno virtual de Python**.

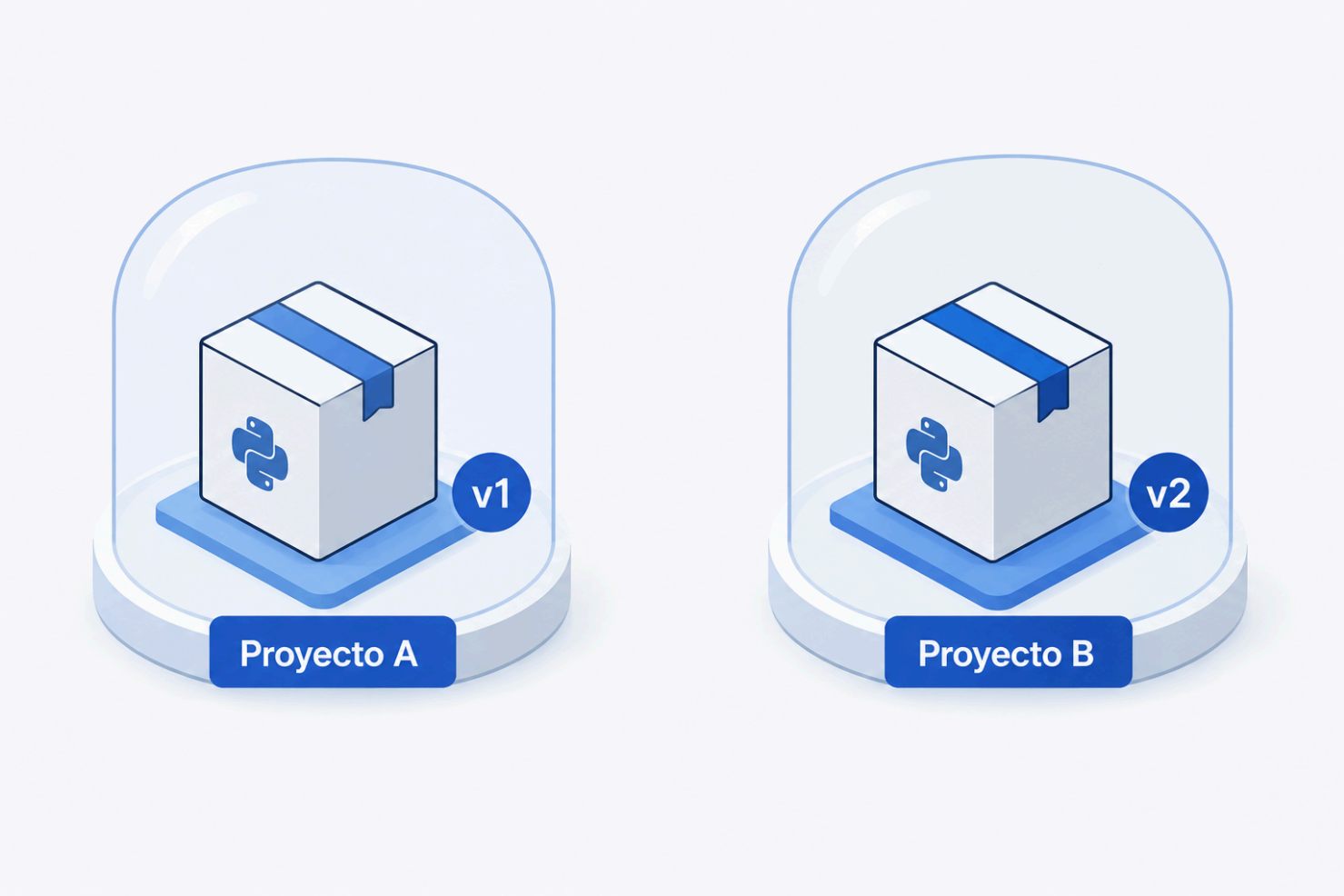
En el **Bloque 2** lo viste de pasada, como parte del recorrido completo. Aquí lo profundizas, porque en este mismo bloque te toca crear el tuyo propio y dejarlo funcionando correctamente sobre tu repositorio.

🧩 Qué hace realmente un entorno virtual

Tu computador tiene una instalación de **Python** compartida por todo lo que corras en él. Si instalas una librería directamente ahí, queda disponible para todos tus proyectos, lo cual suena cómodo hasta que dos proyectos distintos necesitan versiones distintas de la misma librería: uno se rompe apenas el otro actualiza algo.

✅ Un **entorno virtual** resuelve esto creando una copia aislada de **Python** y de sus paquetes, exclusiva para un proyecto.

Cuando lo activas, los comandos de **Python** y de instalación de paquetes dejan de apuntar a la instalación compartida del sistema y apuntan a esa copia aislada, que vive dentro de una carpeta de tu propio repositorio.



Piensa en un caso concreto para que la idea quede clara. Imagina que en un mismo computador tienes dos proyectos:

- uno que ya construiste hace un tiempo y que depende de una versión antigua de una **librería** para hacer peticiones HTTP,
- y otro nuevo (como el que vas a construir en este programa) que necesita una versión más reciente de esa misma librería, con un comportamiento distinto en algunos casos.

⚠️ Sin **entornos virtuales**, instalar la **versión** nueva para tu **proyecto** nuevo actualiza silenciosamente esa librería para todo el computador, y el proyecto antiguo, que dependía del comportamiento anterior, deja de funcionar sin que hayas tocado ni una línea de su código.

✅ **Con un entorno virtual por proyecto, ese conflicto no puede ocurrir**: cada proyecto tiene su

✅ **Con un entorno virtual por proyecto, ese conflicto no puede ocurrir**: cada proyecto tiene su propia copia de la librería, en la versión que él necesita, sin interferir con ninguna otra.

> Esto significa que cada **proyecto** de este programa (y en general, cada proyecto que hagas de aquí en adelante) debería empezar con su **propio entorno virtual**, creado antes de instalar cualquier dependencia.

Si instalas algo antes de crear y activar el entorno, esa dependencia queda en la instalación compartida del sistema, no en tu proyecto, y no vas a poder reproducirla de forma confiable más adelante.

Vas a trabajar sobre el mismo repositorio que inicializaste en la lección anterior de esta unidad: ábrelo en VS Code y usa su terminal integrada para todo lo que sigue.

🛠️ Créalo y actívalo, paso a paso

El comando para crear el entorno es el mismo en cualquier sistema operativo; el que lo activa cambia:

```
# macOS / Linux / WSL
python3 -m venv venv
source venv/bin/activate

# Windows (PowerShell)
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Si en Windows el segundo comando te da un error de política de ejecución de scripts, no es un problema de tu entorno: es una política de seguridad de Windows, y la solución está en la guía de troubleshooting de la siguiente lección de esta unidad.

📦 Instalar dependencias y dejar registro exacto de lo que usaste

Con el entorno activo, cada librería que instalas queda disponible solo dentro de ese entorno.

> ☝️ **Pero instalarla no basta**: también tienes que dejar registro de qué instalaste, para que tu proyecto sea reproducible.

Ese registro va en el archivo **"requirements.txt"** de tu repositorio (ya viene en la plantilla base, aunque vacío). Cada vez que instalas una dependencia nueva, agrégala ahí, con su nombre y, cuando sea

relevante, su versión exacta.

Los entornos virtuales tienen, además, una forma de generar automáticamente la lista completa de todo lo que tienen instalado en ese momento, lista para copiar directo dentro de **"requirements.txt":** no hace falta que la armes a mano, línea por línea, cada vez que agregas algo nuevo.

En términos concretos:

```
pip install nombre_de_la_librería
pip freeze > requirements.txt
```

La razón para fijar versiones exactas, y no solo el nombre de la **librería**, es que una actualización de una dependencia puede cambiar su comportamiento sin avisarte, y romper algo que ayer funcionaba.

Registrar la versión exacta que usaste es lo que le permite a otra persona, o a ti mismo en otro computador, reconstruir tu entorno exactamente como lo tenías, instalando todo lo que aparece en **"requirements.txt"** con un solo comando.



🔵 Este mismo principio (aislar y fijar de forma exacta) es el que vas a volver a encontrar más adelante, a otra escala: cuando en el **Módulo 3** contenerices tu aplicación con Docker, lo que vas a hacer es exactamente esto, pero empaquetando todo el sistema operativo y no solo tus dependencias de **Python**.

Lo que aprendes bien aquí, con un **entorno virtual**, no se vuelve a explicar desde cero cuando llegue

Lo que aprendes bien aquí, con un **entorno virtual**, no se vuelve a explicar desde cero cuando llegue ese momento.

🔎 Cómo saber si tu entorno virtual está realmente activo

Esta confusión es más común de lo que parece, sobre todo si tu computador tiene más de una versión de **Python** instalada (algo frecuente en **Windows**, y también posible en macOS si en algún momento instalaste **Python** por fuera del que trae el sistema).

En ese escenario, es fácil creer que activaste tu **entorno virtual** cuando en realidad la terminal sigue usando una instalación de **Python** completamente distinta, y cualquier dependencia que instales termina en el lugar equivocado sin que aparezca ningún mensaje de error evidente.

Uno de los errores más comunes al empezar es, justamente, **instalar algo creyendo que el entorno estaba activo** cuando en realidad no lo estaba. Antes de instalar cualquier cosa, verifica alguna de estas señales:

✓

Mira tu terminal

El nombre del entorno aparece al inicio de la línea en tu terminal, entre paréntesis, antes de tu usuario y carpeta.

✓

Revisa VS Code

El intérprete de Python que aparece seleccionado en VS Code (visible en la esquina inferior derecha del editor) apunta a una ruta dentro de la carpeta de tu entorno virtual, no a una instalación general del sistema.

✓

Comprueba el ejecutable

Al preguntarle a la terminal dónde está el ejecutable de Python que se está usando, la ruta que responde queda dentro de la carpeta de tu proyecto, no en una carpeta general del sistema operativo.

En términos concretos, la primera señal se ve así en tu terminal:

```
(venv) usuario@computador:~/tu-repositorio$
```

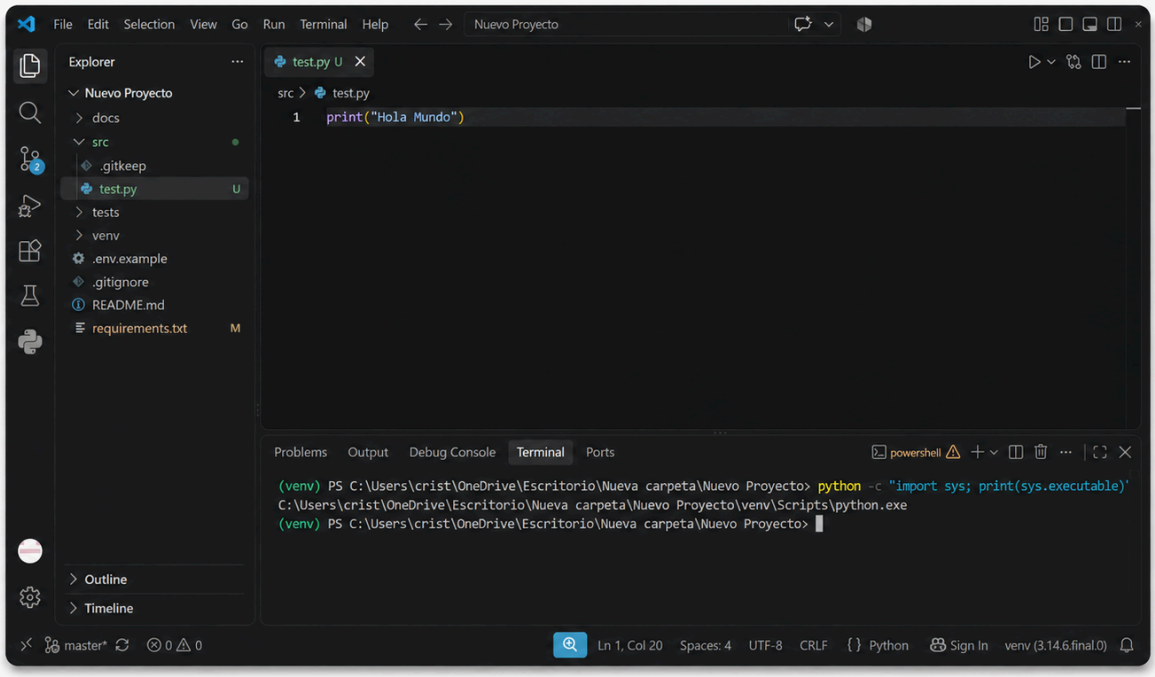
Y la tercera señal, preguntándole a la terminal dónde está el ejecutable, se ve así:

```
# macOS / Linux / WSL
which python3
# /ruta/a/tu/repositorio/venv/bin/python3

# Windows (PowerShell)
where python
# C:\ruta\a\tu\repositorio\venv\Scripts\python.exe
```

🚨 **Si ninguna de estas tres señales calza, tu entorno no está activo todavía**: actívalo antes de instalar nada, o vas a terminar instalando dependencias en el lugar equivocado.

Entorno Virtual Activado



⚠️ Errores comunes con entornos virtuales

Instalar dependencias antes de activar el entorno

Quedan en la instalación general del sistema, no en tu proyecto, y no aparecen en tu **requirements.txt**.

Commitear la carpeta del entorno virtual al repositorio

Es una carpeta pesada, específica de tu computador, y no debería subirse nunca: por eso ya viene excluida en el **.gitignore** de la plantilla base.

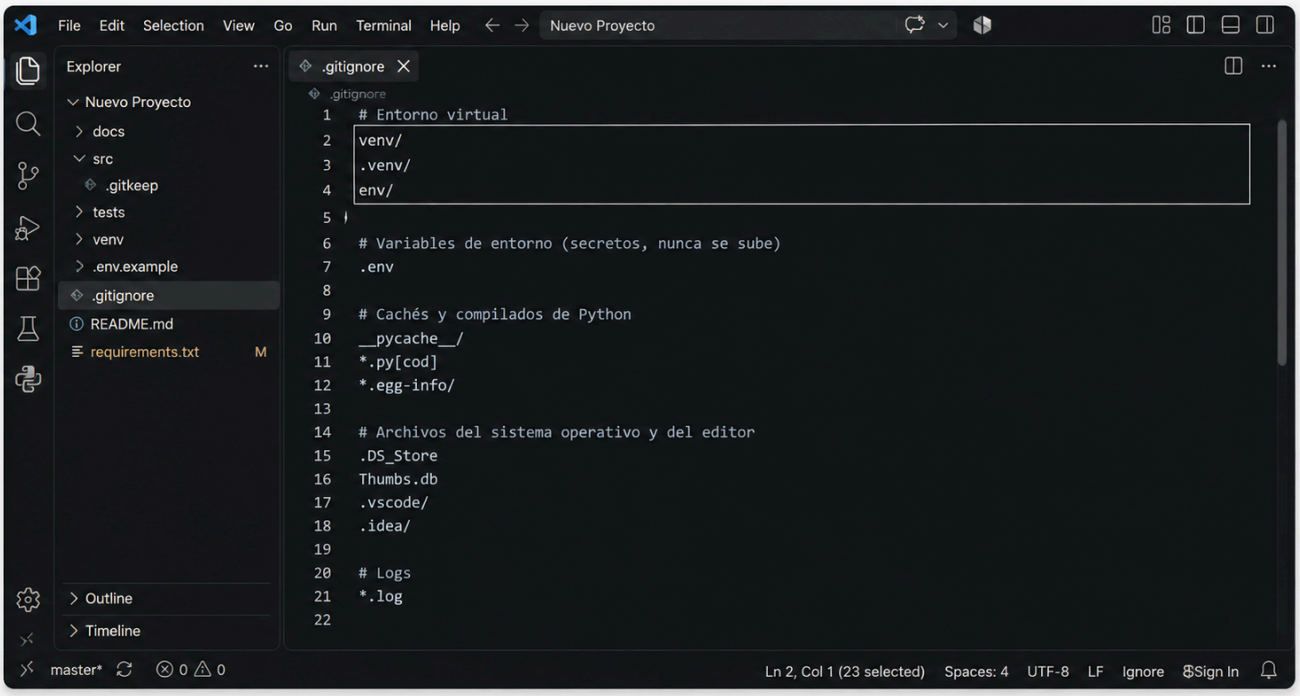
Olvidar actualizar "requirements.txt" al instalar algo nuevo

Si no lo registras ahí, nadie más (ni tú mismo más adelante) va a poder reconstruir tu entorno completo.

Compartir un mismo entorno virtual entre varios proyectos distintos

Elimina justamente la ventaja de aislamiento que buscas: si dos proyectos usan el mismo entorno, vuelves a exponerte al problema original de versiones que compiten entre sí.

Entorno Virtual Excluida



Nada de esto es difícil una vez que lo internalizas, pero sí es fácil pasarlo por alto las primeras veces,

precisamente porque cuando algo sale mal por esta razón, el error que aparece rara vez dice "**tu entorno virtual no está activo**": suele verse como una dependencia que falta, una versión incompatible, o un comando que "**no se reconoce**".

Buena parte de saber depurar problemas de entorno es, en el fondo, aprender a sospechar de esta causa antes que de otras.



Siguiente lección

Bloque 3 · La guía para cuando algo falla

Windows → **macOS** → **WSL**

Con tu **entorno virtual** ya funcionando sobre tu propio repositorio, la siguiente lección de este mismo bloque cubre qué hacer cuando algo de todo este proceso (**terminal**, **Git**, **entorno virtual** o **variables de entorno**) falla, según el **sistema operativo** con el que estés trabajando: los síntomas y las soluciones no son exactamente los mismos en **Windows**, **macOS** o **WSL**.

Última modificación: jueves, 24 de septiembre de 2026, 11:03